

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования**

«Московский технический университет связи и информатики»

Кафедра Программная инженерия

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Введение в информационные технологии»

Тема: «Работа с классами, часть 3»

Выполнил: студент группы БВТ2504

Гапеев Алексей Денисович

Проверил: Харрасов К.Р.

Москва, 2025

Цель работы

Разработать систему управления сотрудниками, демонстрирующую множественное наследование, инкапсуляцию и полиморфизм в Python. Система должна уметь обрабатывать различные типы сотрудников, включая менеджеров и технических специалистов, а также предоставлять возможность для расширения и добавления новых ролей.

Индивидуальное задание

1. Создайте класс **Employee** с общими атрибутами, такими как **name** (имя), **id** (идентификационный номер) и методами, например, **get_info()**, который возвращает базовую информацию о сотруднике.
 2. Создайте класс **Manager** с дополнительными атрибутами, такими как **department** (отдел) и методами, например, **manage_project()**, символизирующим управление проектами.
 3. Создайте класс **Technician** с уникальными атрибутами, такими как **specialization** (специализация), и методами, например, **perform_maintenance()**, означающим выполнение технического обслуживания.
 4. Создайте класс **TechManager**, который наследует как **Manager**, так и **Technician**. Этот класс должен комбинировать управленческие способности и технические навыки, например, иметь методы для управления проектами и выполнения технического обслуживания.
 5. Добавьте метод **add_employee()**, который позволяет **TechManager** добавлять сотрудников в список подчинённых.
 6. Реализуйте метод **get_team_info()**, который выводит информацию о всех подчинённых сотрудниках.
-

7. Создайте объекты каждого класса и демонстрируйте их функциональность.

Скриншоты выполнения

```
/usr/bin/python3.13 /extra/python_projects/Лабораторная 7/main.py
('employee1', 1)
('manager1', 2)
[{'ID': 4, 'name': 'name', 'department': 'department', 'specialization': 'spec'}, {'ID': 5, 'name': 'Name2', 'department': 'Department2', 'specialization': 'Specialization2'}]
Process finished with exit code 0
```

Исходный код программы

class Employee:

```
def __init__(self, identification_num=None, name=None):
    self.name = name
    self.__identification_num = identification_num
```

```
def get_info(self):
    return self.name, self.__identification_num
```

class Manager(Employee):

```
def __init__(self, name, identification_num, department):
    self.department = department
    super().__init__(name, identification_num)
```

```
def manage_project(self):
    pass
```

class Technician:

```
def __init__(self, specialization):
    self.specialization = specialization
```

```
def perform_maintenance(self):
    pass
```

class TechManager(Manager, Technician):

```
def __init__(self, identification_num, name, department, specialization):
    Manager.__init__(self, name, identification_num, department)
    Technician.__init__(self, specialization)
    self.list = []
```

```
def manage_project(self):
    pass
```

```
def perform_maintenance(self):
```

```

pass

def add_employee(self, identification_num, name, department, specialization):
    user_data = {'ID': identification_num, 'name': name, 'department': department,
'specialization': specialization}
    self.list.append(user_data)

def get_team_info(self):
    return self.list

emp = Employee(1, "employee1")
print(emp.get_info())

man = Manager(2, "manager1", "dep1")
print(man.get_info())
man.manage_project()

tech = Technician("spec1")
tech.perform_maintenance()

techman = TechManager("John", 3, "Couch", "")
techman.add_employee(4, "name", "department", "spec")
techman.add_employee(5, "Name2", "Department2", "Specialization2")
print(techman.get_team_info())

```

```

class Employee:

```

```

    def __init__(self, identification_num = None, name = None):
        self.name = name
        self.__identification_num = identification_num

    def get_info(self):
        return self.name, self.__identification_num

```

```

class Manager(Employee):

```

```

    def __init__(self, name, identification_num, department):
        self.department = department
        super().__init__(name, identification_num)

    def manage_project(self):
        pass

```

```

class Technician:

```

```

    def __init__(self, specialization):

```

```

        self.specialization = specialization

    def perform_maintenance(self):
        pass

class TechManager(Manager, Technician):

    def __init__(self, identification_num, name, department, specialization):
        Manager.__init__(self, name, identification_num, department)
        Technician.__init__(self, specialization)
        self.list = []

    def manage_project(self):
        pass

    def perform_maintenance(self):
        pass

    def add_employee(self, identification_num, name, department, specialization):
        user_data = {'ID': identification_num, 'name': name, 'department': department,
'specialization': specialization}
        self.list.append(user_data)

    def get_team_info(self):
        return self.list

emp = Employee(1, "employee1")
print(emp.get_info())

man = Manager(2, "manager1", "dep1")
print(man.get_info())
man.manage_project()

tech = Technician("spec1")
tech.perform_maintenance()

techman = TechManager("John", 3, "Couch", "")
techman.add_employee(4, "name", "department", "spec")
techman.add_employee(5, "Name2", "Department2", "Specialization2")
print(techman.get_team_info())

```

Заключение

В результате выполнения лабораторной работы в программе реализованы методы множественного наследования, инкапсуляции и полиморфизма в Python.